1 — The Neuron (Intro)

A **neuron** (artificial) is the basic computational unit of a neural network.

Mathematically a neuron computes:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$a = \phi(z)$$

- **x** = input vector (features)
- **w** = weight vector (parameters)
- **b** = bias (parameter)
- **z** = linear combination (pre-activation)
- **φ(·)** = activation function → output **a** (post-activation)

Common activations:

- Linear: φ(z)=z (used in regression output)
- Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$ (binary probability)
- ReLU: $\max(0, z)$ (hidden layers)
- Softmax: normalized multi-class outputs.

Intuition: weights control how strongly inputs influence the neuron; activation introduces nonlinearity so networks can model complex functions.

2 — Fitting a Line (Linear Regression) — Theory + Code

2.1 Theory (least squares)

Given training pairs (x_i, y_i), linear model $y = \theta_0 + \theta_1 x$.

Closed form (OLS) for θ = [θ0, θ1]:

$$\boldsymbol{\theta} = (X^\top X)^{-1} X^\top y$$

where X is matrix with a column of ones and column of x values.

2.2 Gradient Descent (1D example)

Loss (MSE):

$$L(\theta) = \frac{1}{N} \sum_i (y_i - \theta_0 - \theta_1 x_i)^2$$

Gradients:

$$\frac{\partial L}{\partial \theta_0} = -\frac{2}{N} \sum (y_i - \hat{y}_i)$$

$$\frac{\partial L}{\partial \theta_1} = -\frac{2}{N} \sum x_i (y_i - \hat{y}_i)$$

Update (learning rate η):

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

python

Copy code

```python
# Fitting a line: closed-form and gradient descent
import numpy as np

# synthetic data
np.random.seed(0)
x = np.linspace(0, 10, 50)
y = 2.5 * x + 1.0 + np.random.randn(len(x)) * 2.0

# Closed form OLS
X = np.vstack([np.ones_like(x), x]).T  # shape (N,2)
theta_closed = np.linalg.inv(X.T @ X) @ X.T @ y
print("Closed-form theta:", theta_closed)

# Gradient descent
theta = np.array([0.0, 0.0])
lr = 0.001
for epoch in range(5000):
    y_pred = X @ theta
    error = y_pred - y
    grad = (2 / len(x)) * (X.T @ error)
    theta = theta - lr * grad
print("GD theta:", theta)
```

3 — Classification: Code Preparation (Checklist + key steps)

Before training classification model (text or numeric):

1. **Data collection & labels** — balanced classes if possible.
2. **Cleaning / normalization** — lowercasing, remove URLs, punctuation, expand contractions optionally.
3. **Tokenization** — split into tokens (word / subword).
4. **Vectorization** — one-hot, TF–IDF, count vectors, or embeddings. For deep models: use `TextVectorization` or pretrained embeddings.
5. **Train–validation–test split** — typical 70/15/15 or 80/10/10.
6. **Batching/shuffling** — for SGD-based optimizers.
7. **Label encoding** — integers for classes (sparse categorical) or one-hot.
8. **Model architecture** — choose appropriate output activation:
   - binary → single unit + sigmoid + binary_crossentropy
   - multi-class → units = K + softmax + categorical_crossentropy
9. **Metrics** — accuracy, precision, recall, F1, confusion matrix.
10. **Early stopping / checkpointing / learning rate schedule.**

4 — Text Classification in TensorFlow (Complete runnable example)

This is a compact **end-to-end** example using `tf.keras`. It uses `TextVectorization`, an Embedding layer, GlobalAveragePooling, and a Dense classifier. Works for binary or multi-class tasks.

python Copy code

```python
# TensorFlow text classification - minimal runnable example
# pip install tensorflow --upgrade

import tensorflow as tf
import numpy as np

# Example toy dataset
texts = [
    "I love this movie, it's amazing!",
    "Terrible movie. I hated it.",
    "An excellent film with great acting",
    "Not good — I will not recommend",
    "Fantastic plot and superb cast",
    "Waste of time, very boring"
]
labels = [1, 0, 1, 0, 1, 0]  # binary: 1=positive, 0=negative

# Split
texts = np.array(texts)
labels = np.array(labels)
idx = np.arange(len(texts))
np.random.shuffle(idx)
texts = texts[idx]; labels = labels[idx]
train_texts, train_labels = texts[:4], labels[:4]
val_texts, val_labels = texts[4:5], labels[4:5]
test_texts, test_labels = texts[5:], labels[5:]

# TextVectorization layer
max_vocab = 2000
max_len = 50
vectorize_layer = tf.keras.layers.TextVectorization(
```

Copy code

```
max_len = 50
vectorize_layer = tf.keras.layers.TextVectorization(
    max_tokens=max_vocab,
    output_mode='int',
    output_sequence_length=max_len
)
# adapt to training texts
vectorize_layer.adapt(train_texts)

# Build model
embedding_dim = 50
model = tf.keras.Sequential([
    tf.keras.Input(shape=(1,), dtype=tf.string),  # input is raw strin
    vectorize_layer,                              # tokenizes -> ints
    tf.keras.layers.Embedding(input_dim=max_vocab, output_dim=embeddin
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(1, activation='sigmoid')  # binary
])

model.compile(loss='binary_crossentropy',
              optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
              metrics=['accuracy'])

model.summary()

# Train
history = model.fit(train_texts, train_labels,
                    validation_data=(val_texts, val_labels),
                    epochs=20, batch_size=2, verbose=2)

# Evaluate
print("Test evaluation:")
print(model.evaluate(test_texts, test_labels, verbose=0))

# Predict
preds = (model.predict(["I liked the film a lot"]) > 0.5).astype("int32
print("Prediction example:", pred
```

5 — The Neuron (revisited) — how a model learns (intuitive + math)

5.1 Forward pass

Given inputs x and parameters θ (weights & biases), compute model prediction $\hat{y} = f_\theta(x)$. For a network, this is repeated for each layer.

5.2 Loss function

Loss measures how far predictions are from targets. Examples:

- Regression → MSE: $L = \frac{1}{N}\sum(y - \hat{y})^2$
- Classification → cross-entropy: $L = -\sum y \log \hat{y}$

5.3 Backpropagation (compute gradients)

We compute gradients of loss w.r.t parameters using chain rule:

$$\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \theta}$$

Automatic differentiation (TensorFlow/PyTorch) computes these efficiently across layers.

5.4 Parameter update (optimizer)

Using gradient descent or variants (SGD, Adam):

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

Optimizers like **Adam** use adaptive moments to accelerate convergence.

5.5 Repeat

Train over many mini-batches and epochs until convergence. Use validation set for early stopping.

6 — Intuition: Why training works

- The loss surface guides parameter movement: gradients point toward directions that reduce loss.
- Activation nonlinearity + multiple layers let networks represent complex, non-linear functions.
- With enough capacity and data, gradient-based optimization finds parameters that generalize (with regularization to avoid overfitting).

---

7 — Extra: Classification code tips & debugging checklist

- **Sanity checks**: train on tiny subset — model should overfit (loss→0).
- **Learning rate**: too high → divergence; too low → very slow.
- **Batch size**: affects stability & speed.
- **Class imbalance**: use class weights or resampling.
- **Metrics**: use confusion matrix, precision/recall when accuracy misleading.
- **Reproducibility**: set seeds for numpy / tf; fix `TF_DETERMINISTIC_OPS` if needed.

---

8 — Short exam answers you can write verbatim

What is a neuron?
An artificial neuron computes a linear combination $z = w^T x + b$ and applies an activation $a = \phi(z)$; weights and bias are learned to map inputs to outputs.

How does a model learn?
By minimizing a loss function via gradient descent: compute forward pass → compute loss → compute gradients (backprop) → update parameters (optimizer). Repeat until loss converges.

Fitting a line:
Use OLS closed form $\theta = (X^\top X)^{-1} X^\top y$ or gradient descent updates $\theta \leftarrow \theta - \eta \nabla_\theta L$.

1 — Text preprocessing: goals & checklist (short)

Goals: clean raw text, normalize tokens, reduce noise, and produce numeric input for models.

Typical steps (in order):

- Lowercase
- Remove/normalize URLs, emails, numbers
- Remove HTML tags
- Normalize Unicode & whitespace
- Expand contractions (optional)
- Remove or keep punctuation depending on task
- Tokenize (word / subword / character)
- Remove stopwords (optional)
- Lemmatize or stem (optional)
- Handle rare words → replace by `<UNK>`
- Build vocabulary (word → id)
- Convert tokens → integer sequences and pad/truncate
- (For ML) compute TF–IDF or other features; (for DL) use embeddings or vectorization layers

python

Copy code

```python
# simple preprocessing utilities
import re
import unicodedata
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt', quiet=True)

CONTRACTIONS = {
    "don't":"do not","can't":"cannot","i'm":"i am","it's":"it is","won
    # add more as needed
}

def clean_text(text):
    # normalize unicode
    text = unicodedata.normalize("NFKC", text)
    # lower
    text = text.lower()
    # remove URLs and emails
    text = re.sub(r'https?://\S+|www\.\S+', ' ', text)
    text = re.sub(r'\S+@\S+', ' ', text)
    # expand contractions (simple)
    for k,v in CONTRACTIONS.items():
        text = re.sub(r'\b'+re.escape(k)+r'\b', v, text)
    # remove non-alphanumeric (keep basic punctuation if needed)
    text = re.sub(r'[^a-z0-9\s]', ' ', text)
    # collapse whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def tokenize(text):
    return word_tokenize(text)

# Example
txt = "I'm learning NLP! Visit http://example.com or mail me at a@b.co
```

```
# Example
txt = "I'm learning NLP! Visit http://example.com or mail me at a@b.com
clean = clean_text(txt)
print(clean)
print(tokenize(clean))
```

Notes & exam points: mention why expanding contractions matters, why you may keep punctuation (sentiment tasks), and that replacement by `<UNK>` for low-frequency tokens reduces vocabulary size.

---

3 — Text preprocessing in TensorFlow (TextVectorization + tf.data)

`TextVectorization` simplifies many steps (lowercase, split, vocab) and integrates with `tf.data` and Keras models.

python

```
import tensorflow as tf

# Example corpus
texts = ["I love neural nets.", "This movie is good.", "Terrible servic

# TextVectorization layer
max_tokens = 10000
output_seq_len = 20

vectorize_layer = tf.keras.layers.TextVectorization(
    max_tokens=max_tokens,
    standardize='lower_and_strip_punctuation',  # builtin standardizer
    split='whitespace',
    output_mode='int',  # 'int' for sequences
    output_sequence_length=output_seq_len
)

# adapt to data (build vocabulary)
vectorize_layer.adapt(texts)
```

```
# adapt to data (build vocabulary)
vectorize_layer.adapt(texts)

# usage
example = tf.constant(["I love this!"])
print(vectorize_layer(example))  # token ids

# build tf.data pipeline
dataset = tf.data.Dataset.from_tensor_slices((texts, [1,1,0]))
dataset = dataset.shuffle(10).batch(2).map(lambda x,y: (vectorize_layer
for x,y in dataset.take(1):
    print(x.shape, y)
```

Copy code

Key points to write in exam:

- `TextVectorization.adapt()` builds vocab from data.
- `output_mode='int'` gives sequences; `'tf-idf'` gives TF–IDF vectors.
- Integrates seamlessly into Keras models (you can include the layer as the first layer).

4 — Word Embeddings — what, why, types

Definition: map discrete tokens → dense real vectors so semantically similar words have similar vectors.

Why: dense embeddings reduce dimensionality, capture semantics, and are efficient for neural models.

Main types

1. **One-hot vectors**
   - Sparse, high-dimensional (vocab size), no notion of similarity.
   - Not used directly for semantic tasks.
2. **Static dense embeddings (context-independent)**
   - Word2Vec (CBOW, Skip-gram), GloVe, FastText.
   - Each word has a single vector regardless of context.
   - Pros: cheap, pre-trained vectors available; cons: cannot handle polysemy.
3. **Subword embeddings** ↓

3. **Subword embeddings**
    - FastText: uses char n-grams — helps with morphology and OOV by composing vectors.
4. **Contextual embeddings (context-dependent)**
    - Transformers (BERT, GPT, RoBERTa): produce different vectors depending on entire sentence/context.
    - Superior for many modern NLP tasks.
5. **Learned embeddings in models**
    - Embedding layer in Keras: initialized randomly and learned end-to-end.

How to use

- **Pretrained** (GloVe, Word2Vec): load vectors, build embedding matrix aligned to your vocabulary, and either freeze or fine-tune.
- **Train from scratch**: use Keras Embedding layer, requires larger datasets.

Objective (Word2Vec style)

- CBOW: predict target word from context vectors.
- Skip-gram: predict context words from a target word.
- Training via negative sampling or hierarchical softmax.

---

5 — CBOW (Continuous Bag of Words) — theory & math

Intuition: CBOW predicts a target word given its surrounding context words (bag-of-words: order not used). It learns word vectors so that the average (or sum) of context vectors is predictive of the target vector.

Setup:

- Window size = `m` -> context = `m` words to left + `m` words to right (often symmetric), total context size `2m`.
- For each training example: context words `c1, c2, ..., cK` and target `w`.
- Model: embed each context word → vectors `v_c1 ... v_cK`. Compute context representation `h = (1/K) * sum(v_ci)` (or sum). Predict `w` with softmax over vocabulary:

↓

- Model: embed each context word → vectors `v_c1 ... v_cK`. Compute context representation `h = (1/K) * sum(v_ci)` (or sum). Predict `w` with softmax over vocabulary:

$$P(w \mid context) = \frac{\exp(u_w^\top h)}{\sum_{w'} \exp(u_{w'}^\top h)}$$

where `u_w` is output vector for word `w` (sometimes same as embedding matrix, sometimes separate).

Loss: cross-entropy (negative log-likelihood):

$$L = -\log P(w \mid context)$$

Optimizations: computing full softmax is expensive for large vocabularies → use **negative sampling** or **hierarchical softmax**.

CBOW vs Skip-gram:

- CBOW: averages context to predict center → faster, good for frequent words.
- Skip-gram: predicts surrounding words from center → works better for infrequent words.

---

6 — CBOW implementation choices & notes

- **Context representation:** average or sum of context embeddings (averaging normalizes by context length).
- **Output parametrization:** use separate output embeddings (`u`) and input embeddings (`v`) or tie them.
- **Negative sampling:** sample `k` negative words per positive example and use binary classification objective for each pair.
- **Window size:** small (2–5) typical.
- **Subsampling frequent words:** downsample very frequent tokens to improve training.
- **Batching & tf.data:** use efficient pipelines; generate training pairs beforehand or on the fly.

7 — CBOW in TensorFlow — runnable example (small toy dataset)

This example shows a minimal CBOW with full softmax (small vocab). For production (large vocab) use negative sampling with custom loss for speed.

python Copy code

```python
# Minimal CBOW implementation in TensorFlow (full softmax) - for small
import tensorflow as tf
import numpy as np
from collections import Counter
import itertools

# 1. toy corpus (replace with larger corpus)
corpus = [
    "the quick brown fox jumps over the lazy dog",
    "i love natural language processing",
    "the dog is quick and smart",
    "natural language understanding and processing"
]

# 2. preprocessing: tokenize and build vocab
def preprocess(text):
    return text.lower().split()

tokens = list(itertools.chain.from_iterable(preprocess(s) for s in cor
vocab_counts = Counter(tokens)
vocab = [w for w,_ in vocab_counts.most_common()]
vocab_size = len(vocab)
word2idx = {w:i for i,w in enumerate(vocab)}
idx2word = {i:w for w,i in word2idx.items()}

# 3. generate CBOW pairs (context -> target)
window_size = 2  # 2 left + 2 right -> context size up to 4
pairs = []
for sent in corpus:
    words = preprocess(sent)
    for i, w in enumerate(words):
        context = []
```

```
        context = []
        for j in range(i-window_size, i+window_size+1):
            if j != i and 0 <= j < len(words):
                context.append(word2idx[words[j]])
        if len(context) > 0:
            pairs.append((context, word2idx[w]))

print("Total training pairs:", len(pairs))

# 4. prepare batches (pad contexts to same length)
max_context_len = 2*window_size
def pad_context(c):
    # pad with -1 which we will mask
    c = list(c) + [-1]*(max_context_len - len(c))
    return c

X_context = np.array([pad_context(c) for c,_ in pairs], dtype=np.int32)
y_target  = np.array([t for _,t in pairs], dtype=np.int32)

# 5. Model: Embedding matrix (input), average context embeddings -> der
embedding_dim = 50

class CBOW(tf.keras.Model):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.vocab_size = vocab_size
        self.emb = tf.keras.layers.Embedding(vocab_size, embedding_dim,
        self.dense = tf.keras.layers.Dense(vocab_size)  # full softmax

    def call(self, inputs):
        # inputs: (batch, context_len) with -1 paddings
        # mask paddings
        mask = tf.cast(tf.not_equal(inputs, -1), tf.float32)  # 1 for r
        # replace -1 with 0 for safe embedding lookup (we will divide b
        safe_inputs = tf.where(inputs==-1, tf.zeros_like(inputs), input
        embeds = self.emb(safe_inputs)  # (batch, context_len, embeddin
        mask_exp = tf.expand_dims(mask, -1)  # (batch, context_len, 1)
        summed = tf.reduce_sum(embeds * mask_exp, axis=1)  # (batch, em
        denom = tf.maximum(tf.red ↓ _sum(mask, axis=1, keepdims=True),
        avg = summed / denom  # (batch, embedding_dim)
```

```
        mask_exp = tf.expand_dims(mask, -1)  # (batch, cont
        summed = tf.reduce_sum(embeds * mask_exp, axis=1)  # (batch, em
        denom = tf.maximum(tf.reduce_sum(mask, axis=1, keepdims=True),
        avg = summed / denom  # (batch, embedding_dim)
        logits = self.dense(avg)  # (batch, vocab_size)
        return logits

model = CBOW(vocab_size, embedding_dim)
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_l
              metrics=['sparse_categorical_accuracy'])

# 6. train
model.fit(X_context, y_target, epochs=200, batch_size=8, verbose=0)

# 7. inspect embeddings
emb_matrix = model.emb.get_weights()[0]  # shape (vocab_size, embedding
# compare nearest neighbors for a word
from sklearn.metrics.pairwise import cosine_similarity
def nearest(word, k=5):
    if word not in word2idx:
        return []
    v = emb_matrix[word2idx[word:][0]] if False else emb_matrix[word2id
    sims = cosine_similarity([v], emb_matrix)[0]
    idxs = np.argsort(sims)[::-1][1:k+1]
    return [(idx2word[i], sims[i]) for i in idxs]

print("Vocab:", vocab)
print("Nearest to 'natural':", nearest('natural', 5))
```

Copy code

otes:

- This code uses **full softmax**; acceptable for tiny vocab. For real corpora use **negative sampling**.
- Padding uses `-1` and is masked. For performance, you may pre-pad to fixed context length or generate fixed-length contexts.
- After training, embeddings are in `emb_matrix` and can be used in downstream tasks.

8 — CBOW with Negative Sampling (conceptual)

Negative sampling turns the multiclass softmax into several binary classification problems: for each positive pair (context, target), sample `k` negative target words and train the model to score positive pairs high and negatives low. Loss per pair:

$$\log \sigma(u_w^\top h) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n} [\log \sigma(-u_{w_i}^\top h)]$$

where `σ` is sigmoid and `P_n` is noise distribution (often unigram^0.75). This is the standard Word2Vec training objective and is far more efficient.

Implementing negative sampling in TF requires custom loss or use `tf.keras.layers.Embedding` with `tf.nn.nce_loss` or `tf.nn.sampled_softmax_loss`.

---

9 — Practical tips & exam-ready bullets

- **When to use CBOW vs Skip-gram:** CBOW is faster and better for frequent words; Skip-gram is better for rare words and often yields higher quality vectors.
- **Window size matters:** small windows capture syntactic relationships; larger windows capture topical/semantic relations.
- **Subsampling:** downsample very frequent words (e.g., function words) during pair generation to improve embedding quality.
- **Embeddings in TF:** use `Embedding` layer for end-to-end learning or load pretrained GloVe into embedding weights for transfer learning.
- **Evaluation of embeddings:** intrinsic (word analogies, nearest neighbor) and extrinsic (performance in downstream tasks).